BUILD2HOST — Main JavaScript

Formatted JavaScript source document

```
/* ==================================
   BUILD2HOST — MAIN JAVASCRIPT
   ================================== */
'use strict';
// ==================== UTILS ====================
const qs = (s, el = document) => el.querySelector(s);
const qsa = (s, el = document) => [...el.querySelectorAll(s)];
const prefersReducedMotion = window.matchMedia('(prefers-reduced-motion: reduce)').matches;
// ==================== FOOTER YEAR ====================
const footerYear = qs('#footerYear');
if (footerYear) footerYear.textContent = new Date().getFullYear();
// ==================== SCROLL PROGRESS ====================
const progressBar = qs('#scrollProgress');
function updateProgress() {
  const scrollTop = window.scrollY;
  const docHeight = document.documentElement.scrollHeight - window.innerHeight;
  const pct = docHeight > 0 ? (scrollTop / docHeight) * 100 : 0;
  if (progressBar) progressBar.style.width = `${pct}%`;
}
// ==================== NAVBAR ====================
const navbar = qs('#navbar');
const hamburger = qs('#hamburger');
const mobileMenu = qs('#mobileMenu');
function handleNavbarScroll() {
  if (window.scrollY > 60) {
    navbar?.classList.add('scrolled');
  } else {
    navbar?.classList.remove('scrolled');
  }
}
// Mobile hamburger
hamburger?.addEventListener('click', () => {
  const isOpen = mobileMenu?.classList.contains('open');
  hamburger.classList.toggle('open', !isOpen);
  hamburger.setAttribute('aria-expanded', String(!isOpen));
  mobileMenu?.classList.toggle('open', !isOpen);
  mobileMenu?.setAttribute('aria-hidden', String(isOpen));
});
// Close mobile menu on link click
qsa('.mobile-link, .mobile-cta').forEach(link => {
  link.addEventListener('click', () => {
    hamburger?.classList.remove('open');
    hamburger?.setAttribute('aria-expanded', 'false');
    mobileMenu?.classList.remove('open');
    mobileMenu?.setAttribute('aria-hidden', 'true');
  });
});
// Active nav link
function updateActiveNavLink() {
  const sections = ['services', 'process', 'about', 'team', 'contact'];
  let current = '';
  sections.forEach(id => {
    const el = qs(`#${id}`);
    if (el && window.scrollY >= el.offsetTop - 200) current = id;
  });
  qsa('.nav-link').forEach(link => {
    const href = link.getAttribute('href')?.replace('#', '');
    link.classList.toggle('active', href === current);
  });
}
// ==================== REVEAL ANIMATIONS ====================
function initReveal() {
  if (prefersReducedMotion) {
    qsa('.reveal').forEach(el => el.classList.add('visible'));
    return;
  }
  const io = new IntersectionObserver((entries) => {
    entries.forEach((entry, i) => {
      if (entry.isIntersecting) {
        // Stagger siblings
        const siblings = qsa('.reveal', entry.target.parentElement);
        const idx = siblings.indexOf(entry.target);
        setTimeout(() => {
          entry.target.classList.add('visible');
        }, idx * 80);
        io.unobserve(entry.target);
      }
    });
  }, { threshold: 0.12, rootMargin: '0px 0px -40px 0px' });
  qsa('.reveal').forEach(el => io.observe(el));
}
// ==================== HERO PARALLAX ====================
function initHeroParallax() {
```

```
  if (prefersReducedMotion) return;
  const heroHeading = qs('#heroHeading');
  const heroContent = qs('#heroContent');
  const heroFloat = qs('#heroFloat');
  const heroShape1 = qs('.hero-shape-1');
  const heroShape2 = qs('.hero-shape-2');
  const heroCue = qs('#scrollCue');
  function onHeroScroll() {
    const scrollY = window.scrollY;
    const heroH = qs('#hero')?.offsetHeight || window.innerHeight;
    if (scrollY > heroH) return;
    const progress = scrollY / heroH;
    if (heroContent) {
      heroContent.style.transform = `translateY(${scrollY * 0.25}px)`;
      heroContent.style.opacity = 1 - progress * 1.4;
    }
    if (heroFloat) {
      heroFloat.style.transform = `translateY(${scrollY * -0.1}px)`;
    }
    if (heroShape1) {
      heroShape1.style.transform = `translateY(${scrollY * -0.3}px)`;
    }
    if (heroShape2) {
      heroShape2.style.transform = `translateY(${scrollY * -0.2}px)`;
    }
    if (heroCue) {
      heroCue.style.opacity = Math.max(0, 1 - progress * 5);
    }
  }
  window.addEventListener('scroll', onHeroScroll, { passive: true });
}
// ==================== HERO HEADING LINE ENTRANCE ====================
function initHeroEntrance() {
  const lines = qsa('.hero-line');
  const label = qs('.hero-label');
  const desc = qs('.hero-desc');
  const sub = qs('.hero-sub');
  const actions = qs('.hero-actions');
  const visual = qs('#heroVisual');
  const els = [label, ...lines, sub, desc, actions, visual].filter(Boolean);
  if (prefersReducedMotion) {
    els.forEach(el => {
      el.style.opacity = '1';
      el.style.transform = 'none';
    });
    return;
  }
  // Set initial state
  els.forEach(el => {
    el.style.opacity = '0';
    el.style.transform = 'translateY(32px)';
    el.style.transition = 'opacity 0.7s cubic-bezier(0.16,1,0.3,1), transform 0.7s cubic-bezier(0.16,1,0.3,1)';
  });
  // Animate in sequence
  els.forEach((el, i) => {
    setTimeout(() => {
      el.style.opacity = '1';
      el.style.transform = 'translateY(0)';
    }, 200 + i * 100);
  });
}
// ==================== SERVICES ACCORDION ====================
function initServices() {
  const rows = qsa('.service-row-header');
  rows.forEach(header => {
    const panelId = header.getAttribute('aria-controls');
    const panel = panelId ? qs(`#${panelId}`) : null;
    header.addEventListener('click', () => {
      const isOpen = header.getAttribute('aria-expanded') === 'true';
      // Close all
      rows.forEach(h => {
        h.setAttribute('aria-expanded', 'false');
        const pid = h.getAttribute('aria-controls');
        const p = pid ? qs(`#${pid}`) : null;
        if (p) {
          p.classList.remove('open');
          p.setAttribute('aria-hidden', 'true');
        }
      });
      // Open clicked (if was closed)
      if (!isOpen && panel) {
        header.setAttribute('aria-expanded', 'true');
        panel.classList.add('open');
        panel.setAttribute('aria-hidden', 'false');
      }
    });
    // Keyboard support
    header.addEventListener('keydown', (e) => {
      if (e.key === 'Enter' || e.key === ' ') {
        e.preventDefault();
```

```
        header.click();
      }
    });
  });
}
// ==================== STACKED WORK CARDS ====================
function initStackedCards() {
  if (prefersReducedMotion) return;
  const cards = qsa('.work-card');
  if (!cards.length) return;
  // Subtle scale effect as cards stack
  const io = new IntersectionObserver((entries) => {
    entries.forEach(entry => {
      const card = entry.target;
      const idx = cards.indexOf(card);
      if (entry.isIntersecting) {
        // Scale down previous cards subtly
        cards.slice(0, idx).forEach(prev => {
          prev.classList.add('scaled');
        });
      } else if (entry.boundingClientRect.top > 0) {
        // Unscale when scrolling back up
        card.classList.remove('scaled');
        cards.slice(0, idx).forEach(prev => {
          prev.classList.remove('scaled');
        });
      }
    });
  }, { threshold: 0.3 });
  cards.forEach(card => io.observe(card));
}
// ==================== PROCESS STEP HIGHLIGHT ====================
function initProcess() {
  if (prefersReducedMotion) return;
  const steps = qsa('.process-step');
  const io = new IntersectionObserver((entries) => {
    entries.forEach(entry => {
      if (entry.isIntersecting) {
        entry.target.style.opacity = '1';
        entry.target.style.transform = 'none';
      }
    });
  }, { threshold: 0.4 });
  steps.forEach((step, i) => {
    step.style.opacity = '0';
    step.style.transform = 'translateX(-20px)';
    step.style.transition = `opacity 0.6s ${i * 0.1}s cubic-bezier(0.16,1,0.3,1), transform 0.6s ${i * 0.1}s cubic-
bezier(0.16,1,0.3,1)`;
    io.observe(step);
  });
}
// ==================== TECH TAG ANIMATION ====================
function initTechTags() {
  if (prefersReducedMotion) return;
  const cloud = qs('#techCloud');
  if (!cloud) return;
  const tags = qsa('.tech-tag', cloud);
  tags.forEach(tag => {
    tag.style.opacity = '0';
    tag.style.transform = 'translateY(20px) scale(0.94)';
  });
  const io = new IntersectionObserver((entries) => {
    entries.forEach(entry => {
      if (entry.isIntersecting) {
        tags.forEach((tag, i) => {
          const delay = parseInt(tag.style.getPropertyValue('--delay')) || i * 60;
          setTimeout(() => {
            tag.style.transition = 'opacity 0.5s cubic-bezier(0.16,1,0.3,1), transform 0.5s cubic-bezier(0.16,1,0.
3,1)';
            tag.style.opacity = '1';
            tag.style.transform = 'none';
          }, delay);
        });
        io.unobserve(entry.target);
      }
    });
  }, { threshold: 0.2 });
  io.observe(cloud);
}
// ==================== CONTACT FORM ====================
function initContactForm() {
  const form = qs('#contactForm');
  if (!form) return;
  form.addEventListener('submit', (e) => {
    e.preventDefault();
    const btn = qs('#formSubmit');
    const name = qs('#fname')?.value.trim();
    const phone = qs('#fphone')?.value.trim();
    const email = qs('#femail')?.value.trim();
    const service = qs('#fservice')?.value;
```

```
    const project = qs('#fproject')?.value.trim();
    if (!name || !phone || !email || !service) {
      showFormMessage('Please fill in all required fields.', 'error');
      return;
    }
    if (!/^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(email)) {
      showFormMessage('Please enter a valid email address.', 'error');
      return;
    }
    // Build WhatsApp message (fallback since no backend)
    const msg = `Hi build2host! My name is ${name}.\n\nService: ${service}\nPhone: ${phone}\nEmail: ${email}\n\
nProject: ${project || 'Not specified'}`;
    const waUrl = `https://wa.me/919840264843?text=${encodeURIComponent(msg)}`;
    // Visual feedback
    if (btn) {
      btn.textContent = 'Sending... ✓';
      btn.style.background = '#25d366';
      btn.style.color = '#fff';
    }
    setTimeout(() => {
      window.open(waUrl, '_blank', 'noopener,noreferrer');
      showFormMessage('Request sent! Redirecting to WhatsApp...', 'success');
      form.reset();
      if (btn) {
        btn.innerHTML = 'Send Project Request <span class="btn-arrow">→</span>';
        btn.style.background = '';
        btn.style.color = '';
      }
    }, 600);
  });
}
function showFormMessage(msg, type) {
  let msgEl = qs('#formMessage');
  if (!msgEl) {
    msgEl = document.createElement('p');
    msgEl.id = 'formMessage';
    msgEl.style.cssText = 'margin-top:12px; font-size:0.88rem; padding:12px 16px; border-radius:10px; font-weight:
500;';
    qs('#contactForm')?.appendChild(msgEl);
  }
  msgEl.textContent = msg;
  msgEl.style.background = type === 'error' ? 'rgba(220,50,50,0.15)' : 'rgba(37,211,102,0.15)';
  msgEl.style.color = type === 'error' ? '#ff6b6b' : '#25d366';
  msgEl.style.border = `1px solid ${type === 'error' ? 'rgba(220,50,50,0.2)' : 'rgba(37,211,102,0.2)'}`;
  setTimeout(() => { if (msgEl) msgEl.remove(); }, 5000);
}
// ==================== SMOOTH SCROLL LINKS ====================
function initSmoothScroll() {
  qsa('a[href^="#"]').forEach(link => {
    link.addEventListener('click', (e) => {
      const href = link.getAttribute('href');
      if (!href || href === '#') return;
      const target = qs(href);
      if (!target) return;
      e.preventDefault();
      const navH = navbar?.offsetHeight || 80;
      const targetY = target.getBoundingClientRect().top + window.scrollY - navH - 20;
      window.scrollTo({ top: targetY, behavior: 'smooth' });
    });
  });
}
// ==================== FLOATING BADGE MICRO-INTERACTION ====================
function initBadgeHover() {
  const badges = qsa('.float-badge');
  badges.forEach(badge => {
    badge.addEventListener('mouseenter', () => {
      badge.style.transform = 'scale(1.08) translateY(-4px)';
      badge.style.transition = 'transform 0.25s cubic-bezier(0.16,1,0.3,1)';
    });
    badge.addEventListener('mouseleave', () => {
      badge.style.transform = '';
    });
  });
}
// ==================== SCROLL HANDLER ====================
let ticking = false;
function onScroll() {
  if (!ticking) {
    requestAnimationFrame(() => {
      updateProgress();
      handleNavbarScroll();
      updateActiveNavLink();
      ticking = false;
    });
    ticking = true;
  }
}
// ==================== INIT ====================
document.addEventListener('DOMContentLoaded', () => {
  initReveal();
```

```
  initHeroEntrance();
  initHeroParallax();
  initServices();
  initStackedCards();
  initProcess();
  initTechTags();
  initContactForm();
  initSmoothScroll();
  initBadgeHover();
  window.addEventListener('scroll', onScroll, { passive: true });
  // Initial calls
  updateProgress();
  handleNavbarScroll();
});
```